



Transaction States in DBMS



Transaction & Its States (5 Marks Answer)

◆ Definition

1. A transaction is a sequence of database operations executed as a single logical unit of work.
2. It ensures data integrity and follows **ACID** properties.

◆ States of a Transaction

3. **Active State** – Transaction is executing read/write operations.
4. **Partially Committed State** – All operations are done, but changes are not permanently saved.
5. **Committed State** – Transaction is successfully completed and changes are permanently stored.
6. **Failed State** – Transaction cannot proceed due to error or system failure.
7. **Aborted State** – Transaction is rolled back and database is restored to previous state.
8. **Terminated State** – Transaction ends after commit or abort and releases resources.

◆ Flow of States

9. Normal flow: Active → Partially Committed → Committed → Terminated
10. Failure flow: Active → Failed → Aborted → Terminated

ACID Properties of a Transaction (5 Marks Answer)

Definition

1. ACID properties are a set of rules that ensure reliable and consistent transaction processing in DBMS.

ACID Properties

2. **Atomicity** – A transaction is executed **completely or not at all**. 🍌 Example: If money is debited but not credited, the transaction is rolled back.
3. **Consistency** – A transaction brings the database from **one valid state to another valid state**. 🍌 Example: Total balance before and after transfer remains correct.
4. **Isolation** – Transactions are executed **independently**, without affecting each other. 🍌 Example: Two users updating same data do not interfere.
5. **Durability** – Once a transaction is committed, changes are **permanently stored**, even after system failure. 🍌 Example: After successful transfer, data remains saved even if system crashes.

Summary

6. Atomicity ensures **all or nothing execution**.
7. Consistency ensures **correct data state**.
8. Isolation ensures **no interference between transactions**.
9. Durability ensures **permanent storage of data**.

Conclusion

10. ACID properties ensure **data reliability, integrity, and correctness** in DBMS.

Serializability (5 Marks Answer)

◆ Definition

1. **Serializability** is a property of a schedule where its result is **equivalent to some serial schedule**.
2. It ensures that concurrent transactions produce a **correct and consistent result**.

◆ Explanation

3. In DBMS, transactions often execute concurrently (non-serial).
4. Serializability checks whether such execution gives the **same output as serial execution**.

◆ Importance

5. Ensures **data consistency** in concurrent execution.
6. Prevents problems like **lost update, dirty read, and inconsistency**.
7. Maintains **correctness of database operations**.
8. Allows **parallel execution** while preserving accuracy.
9. Improves system performance without compromising reliability.

Conclusion

10. Serializability is essential to ensure **safe and correct concurrent transaction execution** in DBMS.

View Serializability & Comparison (5 Marks Answer)

Definition

1. **View serializability** is a type of serializability where a schedule is **view equivalent** to a serial schedule.
2. Two schedules are view equivalent if they have same **read-from relation**, **initial read**, and **final write**.

Conditions of View Equivalence

3. Same transaction reads the **initial value** of data item in both schedules.
4. If T_i reads a value written by T_j , same must happen in both schedules.
5. The **final write** on each data item must be by the same transaction.

Comparison with Conflict Serializability

6. **Conflict serializability** is based on **conflicting operations** and **precedence graph**.
7. **View serializability** is based on **final result** and **data dependency**.
8. All conflict serializable schedules are **view serializable**, but not vice versa.
9. View serializability is **more general but difficult to test**, while conflict serializability is easier.

Conclusion

10. View serializability ensures correctness based on **final output**, while conflict serializability uses **operation conflicts** for checking.

◆ Definition

1. **Conflict serializability** is a type of serializability where a schedule can be converted into a **serial schedule by swapping non-conflicting operations**.
2. Two operations conflict if they belong to different transactions and access the **same data item**, and at least one is a write operation.

◆ Conflicting Operations

3. Read–Write (R-W), Write–Read (W-R), and Write–Write (W-W) operations cause conflict.

◆ Precedence Graph

4. A **precedence graph** (serialization graph) is used to check conflict serializability.
5. Nodes represent transactions, and edges represent conflict order ($T_i \rightarrow T_j$).

◆ Condition

6. If the precedence graph has **no cycle**, the schedule is conflict serializable.
7. If there is a **cycle**, the schedule is not conflict serializable.

◆ Example

8. Schedule: R1(X), W1(X), R2(X), W2(X)
9. Conflict: $T_1 \rightarrow T_2$ (edge from T1 to T2), graph has no cycle $\rightarrow$ serializable.

✓ Conclusion

0. Conflict serializability ensures **correct execution of concurrent transactions using precedence graph analysis**.



Types of Schedules (5 Marks Answer)

◆ Definition

1. Different types of schedules define how transactions are executed while maintaining consistency in DBMS.

◆ Types of Schedules

2. **Serializable Schedule** – A schedule that produces the same result as a serial schedule.
3. Ensures correctness and consistency of data.
4. **Non-Serializable Schedule** – Does not produce the same result as a serial schedule.
5. May lead to **data inconsistency**.
6. **Recoverable Schedule** – A transaction commits only after the transaction from which it reads has committed.
7. Ensures safe recovery in case of failure.
8. **Cascadeless Schedule** – Transactions read only committed data.
9. Avoids cascading aborts and improves reliability.

Conclusion

10. These schedules help maintain **data consistency**,  **correctness**, and **safe execution of transactions**.

Cascading Aborts (5 Marks Answer)

◆ Definition

1. **Cascading aborts** occur when failure of one transaction causes **multiple dependent transactions** to rollback.
2. This happens when a transaction reads **uncommitted data** from another transaction.

◆ Explanation

3. If a transaction is aborted, all transactions that used its data must also be aborted.
4. This leads to a **chain reaction of rollbacks**.

◆ Example

5. T1 writes X $\rightarrow$ W1(X)
6. T2 reads X $\rightarrow$ R2(X) (before T1 commits)
7. T1 fails and is aborted
8. T2 must also be aborted because it used uncommitted data

◆ Key Point

9. Cascading aborts can be avoided using **cascadeless schedules**.



Conclusion

10. Cascading aborts reduce system reliability, so DBMS aims to avoid them for **safe transaction execution**.



Difference Between Schedules (5 Marks Answer)

Feature	Recoverable Schedule	Non-Recoverable Schedule	Cascadeless Schedule
Definition	Transaction commits only after the transaction it depends on commits	Transaction commits before the transaction it depends on commits	Transactions read only committed data
Dependency	Allows dependency on uncommitted data	Depends on uncommitted data and commits early	No dependency on uncommitted data
Safety	Safe for recovery	Unsafe, may cause inconsistency	Very safe
Cascading Abort	May occur	May occur	Does not occur
Data Consistency	Maintained	May be violated	Strongly maintained
Example	T2 commits after T1 commits	T2 commits before T1 commits	T2 reads only after T1 commits

Concurrency Control (5 Marks Answer)

◆ Definition

1. **Concurrency control** is a mechanism in DBMS that manages **simultaneous execution of multiple transactions**.
2. It ensures that transactions are executed in a way that maintains **data consistency and integrity**.

◆ Need / Requirement

3. Multiple transactions may access the same data at the same time.
4. Without control, it can lead to **data inconsistency**.
5. Prevents problems like **lost update, dirty read, and uncommitted data access**.
6. Ensures **serializability** of transactions.
7. Maintains **isolation property** of ACID.
8. Allows **safe concurrent execution** of transactions.
9. Improves **system performance and resource utilization**.

Conclusion

10. Concurrency control is required to ensure **correct, consistent, and reliable execution of concurrent transactions** in DBMS.

Lock-Based Protocols (5 Marks Answer)

◆ Definition

1. Lock-based protocols are concurrency control techniques that use **locks to control access** to data items.
2. A transaction must acquire a lock before performing any operation on data.

◆ Types of Locks

3. Shared Lock (S-Lock) – Allows a transaction to **read** a data item.
4. Multiple transactions can hold shared locks on the same data simultaneously.
5. Used for **read operations only**.
6. Exclusive Lock (X-Lock) – Allows a transaction to **read and write** a data item.
7. Only one transaction can hold an exclusive lock on a data item at a time.
8. Prevents other transactions from accessing the data.

◆ Key Points

9. Locks ensure **data consistency and isolation** during concurrent execution.

Conclusion

10. Lock-based protocols control access using shared and exclusive locks to ensure **safe and correct transaction execution**.

Two-Phase Locking (2PL) (5 Marks Answer)

Definition

1. Two-Phase Locking (2PL) is a lock-based concurrency control protocol that ensures serializability.
2. It divides transaction execution into **two phases: growing phase and shrinking phase.**

Phases

3. **Growing Phase** – Transaction can acquire locks but cannot release any lock.
4. **Shrinking Phase** – Transaction can release locks but cannot acquire new locks.

Types of 2PL

5. **Basic 2PL** – Follows growing and shrinking phases.
6. **Strict 2PL** – All exclusive locks are released only after commit/abort.
7. Prevents **cascading aborts.**
8. **Rigorous 2PL** – All locks (shared and exclusive) are released only after commit.
9. Provides stronger isolation than strict 2PL.

Conclusion

10. 2PL ensures **serializable and safe execution of transactions** using controlled locking phases.

Final Simple Summary

READ Rule:

Older cannot read data updated by newer →  abort

WRITE Rule:

- Older cannot write if newer already read →  abort
- Older write ignored if newer already wrote →  ignore

Timestamp-Based Concurrency Control (5 Marks Answer)

◆ Definition

1. Timestamp-based concurrency control is a method where each transaction is assigned a **unique timestamp**.
2. It ensures transactions are executed in **timestamp order** to maintain consistency.

◆ Working

3. Each data item has **Read Timestamp (RTS)** and **Write Timestamp (WTS)**.
4. If a transaction tries to read/write in wrong order, it is **rolled back**.

◆ Rules

5. **Read Rule** – If $TS(T_i) < WTS(X)$, then T_i is aborted (cannot read outdated value).
6. Otherwise, read is allowed and $RTS(X)$ is updated.
7. **Write Rule** – If $TS(T_i) < RTS(X)$, then T_i is aborted.
8. If $TS(T_i) < WTS(X)$, then write is rejected/ignored.

◆ Example

9. T_1 ($TS=5$), T_2 ($TS=10$); if T_2 writes X first, then T_1 tries to read/write $X \rightarrow T_1$ is aborted.



Conclusion

10. Timestamp ordering ensures **conflict-free and serializable execution** based on time order.

◆ Definition

1. A **deadlock** is a situation where two or more transactions are **waiting for each other to release resources**, and none can proceed.
2. It leads to **indefinite waiting** and stops execution of transactions.

◆ Necessary Conditions (Coffman Conditions)

3. **Mutual Exclusion** – Resources cannot be shared; only one transaction can use a resource at a time.
4. **Hold and Wait** – A transaction holds at least one resource and waits for additional resources.
5. **No Preemption** – Resources cannot be forcibly taken; they are released only voluntarily.
6. **Circular Wait** – A circular chain of transactions exists, where each transaction waits for a resource held by another.

◆ Explanation

7. All four conditions must occur simultaneously for a deadlock to happen.
8. If any one condition is removed, deadlock can be avoided.

◆ Example

9. T1 holds resource A and waits for B, while T2 holds B and waits for A → deadlock occurs.

✓ Conclusion



10. Deadlock is a critical problem in concurrency control and must be handled to ensure smooth transaction execution.



Deferred Update vs Immediate Update (5 Marks Answer)



Feature	Deferred Update	Immediate Update
Definition	Changes are recorded in log but applied to database only after commit	Changes are applied to database immediately
Data Update Time	After commit	Before commit
Use of Log	Used for redo only	Used for undo and redo
Recovery	No need to undo uncommitted transactions	Requires undo of uncommitted transactions
Consistency	Database remains unchanged until commit	Database may have uncommitted changes
Complexity	Simple	More complex
Example	Changes applied after successful commit	Changes applied during execution

Shadow Paging & Checkpoints (5 Marks Answer)

◆ Definition

1. **Shadow paging** and **checkpoints** are recovery techniques used to maintain **database consistency** after failure.

◆ Shadow Paging

2. **Shadow paging** maintains two copies: **current page** and **shadow page**.
3. Shadow page is never modified; updates are made on new pages.
4. On commit, new pages replace old pages.
5. On failure, system uses **shadow page** to restore data (no undo/redo needed).

◆ Checkpoints

6. A **checkpoint** is a point where the system saves the **current state of the database**.
7. All log records and modified data are written to stable storage.
8. Reduces recovery time by limiting log scanning.
9. During recovery, system starts from the last checkpoint.

Conclusion

10. Shadow paging avoids undo/redo, while checkpoints improve recovery speed and efficiency.